Prompt 1 — Load + Inspect

```
Write a Python script that loads a CSV file called "data.csv".

Print:
- First 5 rows
- Column names
- Data types

Handle file not found errors gracefully.

Keep code simple and readable.
```

Prompt 2 — Cleaning

```
Extend the script to clean the dataset:

- Drop rows with missing values in:
  county, analyte_name, maximum_concentration, sdwa_limit, year

- Convert:
  maximum_concentration → float
  sdwa_limit → float
  year → integer

- Print:
  - Number of rows before cleaning
  - Number of rows after cleaning

Keep it simple and readable.
```

Prompt 3 — Filter (Arsenic Only)

```
Update the script to filter the dataset:

- Keep only rows where analyte_name == "Arsenic"

Print:
- Number of rows after filtering
- Unique counties remaining
```

Prompt 4 — Aggregation (Core Logic)

```
Add aggregation logic:

For each county:
- Find the MOST RECENT year available
- Filter data to only that year
- Compute:
  - Maximum concentration (worst-case)
  - Average concentration
  - sdwa_limit (use the value for that county)

Store results in a dictionary like:

{
  county: {
    "max_concentration": value,
    "avg_concentration": value,
```

```
    "year": value,
    "sdwa_limit": value
  }
}

Print one example county result.
```

Prompt 5 — Risk Calculation

```
Add risk calculation to each county:

risk_score = min(100, (max_concentration / sdwa_limit) * 100)

Assign category:
- 0–40 → Well Below Limit
- 40–75 → Moderate
- 75–100 → Approaching Limit
- 100 → Above Limit

Add risk_score and category to each county result.
```

Prompt 6 — Confidence Score

```
Add confidence level based on data recency:

- If data is ≤1 year old → "High"
- If 1–3 years old → "Medium"
- If >3 years old → "Low"

Add confidence to each county result.
```

Prompt 7 — Fallback Handling

```
Add fallback handling:

- If max_concentration is missing:
    use avg_concentration instead
    set fallback_used = True
- Otherwise:
    fallback_used = False

Add fallback_used field to each county result.
```

Prompt 8 — Explanation (CRITICAL)

```
Generate a plain English explanation for each county.

Rules:
- If category is "Well Below Limit":
    say levels are well below EPA limits
- If "Moderate":
    say below limits but not negligible
- If "Approaching Limit":
    say close to EPA regulatory limits
- If "Above Limit":
    say exceeds EPA limits
IMPORTANT:
```

```
- Always include:
  "This does NOT mean unsafe water"
- Mention long-term monitoring where appropriate
- Keep explanation short (2–3 sentences max)
- Avoid technical jargon

Add explanation to each county result.
```

Prompt 9 — Final Output Structure

```
Convert results into a list of dictionaries.

Each entry must include:
- county
- risk_score
- category
- max_concentration
- avg_concentration
- explanation
- confidence
- fallback_used

Print one sample output clearly formatted.
```

Prompt 10 — Export JSON

```
Save the final results to a file called "output.json".

Use clean formatting (indent=2).

Print confirmation that the file was saved.
```

Prompt 11 — UI (FINAL)

```
Build a simple static webpage (HTML + CSS + JavaScript) that displays water quality results from "output

Requirements:

1. Load output.json automatically

2. UI:
- Dropdown to select county
- Display:
  - Risk score (large font)
  - Category (color-coded)
  - Max concentration (label: "worst-case measurement")
  - Average concentration (label: "typical level")
  - Explanation (plain English)
  - Confidence level
  - If fallback_used = true, show:
    "Some values estimated due to missing data"

3. Include this note:
"Based on public system-level data, not household-specific measurements"

4. Design:
- Clean, minimal
- Card layout
- Large readable fonts
- Color coding:
```

green / yellow / orange / red

5. Keep everything in ONE HTML file
6. No frameworks

Goal: clear, interpretable UI—not complex visuals.